MVP Feature Scope

The MVP includes several essential components. First, the system must be able to parse a structured scenario file to extract the graph, depot location, list of deliveries, and number of couriers. The program must represent the map as a weighted graph and compute shortest paths using Dijkstra's algorithm. Deliveries should be assigned across couriers (at a minimum using a round-robin rule), and the system must generate a route that connects the depot to each assigned delivery. Finally, the project requires a simulation step that traverses those routes and produces completion timestamps.

In addition to meeting completeness requirements, my project includes supporting multiple routing heuristics, such as fixed-order routing, greedy nearest-neighbor delivery selection, and a route improvement step using a 2-swap local optimization strategy. The project also enables each algorithm and major function to be isolated and testable.

Current Progress

The routing logic and pathfinding components are fully implemented. That includes Dijkstra's algorithm and utility functions for reconstructing and evaluating paths. The delivery assignment and route-ordering logic are also completed, including all three routing strategies planned for the MVP. The scenario file structure and part of the parsing logic are drafted and working for simple cases but still need input validation. The simulation engine exists but needs polished output formatting. The command-line entry point and testing framework still need to be implemented.

Writing Dijkstra's algorithm using Python's heapq went smoothly once the graph format was finalized. The scheduling and round-robin assignment were also fairly easy. The hardest part so far has been the 2-swap optimization logic, because it required iterative improvement and careful checking to avoid unnecessary loops. Designing the modules and keeping responsibilities separate took more planning than expected, but it has made the code easier to manage and extend.

Next Steps

My next priorities are to finish the scenario parser and finalize the simulation output formatting. After that, I will connect all components in main.py so the project can be run from the command line with different scheduling strategies. Once the system runs end-to-end, I will create extra scenario files for testing and write a small automated test script. After testing, I plan to clean up comments, add docstrings, and finalize documentation.